

StrangerTec

Documento Técnico

Emanuel Gutiérrez Moreira

Carné: 2026091141

Francisco Li Ruiz

Carné: 2026014113

Ingeniería en Computadores, Instituto Tecnológico de Costa Rica

Fundamentos de sistemas computacionales

Profesor: Milton Villegas Lemus

12 de mayo de 2026

Introducción:

El proyecto StrangerTec traductor de Morse consiste en el diseño y construcción de un sistema empotrado capaz de interpretar señales en código Morse mediante un pulsador físico, procesarlas y transmitir las de forma inalámbrica a una interfaz gráfica de usuario. El sistema utiliza una Raspberry Pi Pico W (versión 1) como núcleo de procesamiento, la cual traduce los pulsos eléctricos en caracteres alfanuméricos y los envía por Sockets TCP/IP a una aplicación Python en el PC.

La implementación física se basa en una maqueta interactiva inspirada en la serie Stranger Things. Sus componentes principales son: la Raspberry Pi Pico W (v1, pines presoldados), que gestiona la lógica principal, el WiFi y los GPIO; el Regulador de Tensión L7805, que reduce los 9 V de la batería a 5 V estables para proteger la placa; dos registros de corrimiento SN74LS164N, que controlan 13 columnas de LEDs con solo 2 pines GPIO (Data y Clock), operando de forma independiente (7 LEDs en el primero y 6 en el segundo); 16 LEDs jumbo direccionales (rojos), donde 13 representan columnas —conectados a los registros— y 3 representan filas —conectados directamente a pines GP13, GP14 y GP15—; un botón con casquillo en GP16 configurado con PULL_DOWN interno para entrada Morse (presión corta = punto, larga = raya); un buzzer pasivo en GP5 con señal PWM a 1 kHz para retroalimentación auditiva; y una batería de 9 V conectada a un interruptor y al regulador L7805, cuya salida alimenta el pin VSYS (pin 40) de la Raspberry.

En cuanto a la programación del sistema empotrado en MicroPython, se integró el código de conexión WiFi y control de registros aprendidos en los talleres con la lógica propia del proyecto. La función `iniciar_sistema()` contiene el bucle principal: detecta la duración del pulso del botón mediante `time.ticks_ms()` y `time.ticks_diff()` para clasificar puntos (<300 ms) y rayas (>=300 ms), acumula el código Morse y, tras 1,5 s de silencio, lo traduce con el

diccionario MORSE_DICC y actualiza la matriz de LEDs a través de actualizar_maqueta(). La detección de espacios entre palabras se realiza tras 3 s de inactividad. Toda la lógica de comunicación con el PC se maneja mediante Sockets TCP/IP, enviando símbolos y letras en tiempo real al servidor en la computadora.

En el PC, el software StrangerTec_Last.py implementa un servidor multihilo con la librería threading, permitiendo que el hilo principal gestione la interfaz (Tkinter) mientras un hilo secundario mantiene la escucha activa de conexiones entrantes, garantizando comunicación asíncrona. Las librerías utilizadas son: tkinter (con messagebox y ttk) para la interfaz gráfica; PIL para la imagen de fondo; socket para el enlace TCP/IP; threading para la gestión concurrente; random para la selección aleatoria de palabras; y os para resolver rutas de archivos. El código fuente se gestionó con Git en ramas separadas (Desarrollo_1 para la Raspberry y rama de interfaz para el PC), integrando ambas en la rama principal al final del desarrollo.

Conclusiones:

- Se verificó que los dos registros de corrimiento SN74LS164N deben conectarse de forma independiente (cada uno con su propio pin Data y Clock en la Raspberry), y no en cascada como indica el diagrama del proyecto; esta arquitectura simplificó significativamente el código de control y eliminó errores de señal detectados el 30/04/26.
- Se determinó que los pines Q4 (pin 10) del primer registro y los pines Q1 (pin 3) y Q4 (pin 10) del segundo registro SN74LS164N estaban defectuosos de fábrica, lo que obligó a reasignar la columna 5 al pin 13 y la columna 9 al pin 12 del segundo registro. Las pruebas individuales de cada pin antes del montaje final son indispensables.

- El uso de registros de corrimiento SN74LS164N permitió expandir las salidas de la Raspberry Pi Pico W, controlando 13 columnas con solo 2 pines GPIO, optimizando el uso del microcontrolador para otras tareas.
- Se determinó que la caída de tensión en los LEDs Jumbo requiere el uso de resistencias limitadoras ($330\text{--}220\ \Omega$) para proteger las salidas del integrado SN74LS164N y garantizar su operación dentro de los parámetros seguros.
- La implementación del regulador L7805 resultó crítica para estabilizar los 9 V de la batería a los 5 V constantes que requiere el pin VSYS de la Raspberry; sin este componente se arriesga daño permanente del microcontrolador.
- Se comprobó el 1/05/26 que el botón Morse no respondía porque no recibía los 3,3 V; al conectar una pata al pin 3,3VOut de la Raspberry y la otra al GP16, el funcionamiento fue inmediato y estable.
- La configuración PULL_DOWN interna en el pin GP16 del pulsador eliminó el ruido eléctrico, garantizando lecturas digitales limpias y sin falsas detecciones durante las pruebas.
- Se observó que el LED de la fila 2 (GP14) se quemó el 1/05/26; su reemplazo confirmó la importancia de probar cada LED de forma individual antes del ensamblaje final de la maqueta.
- El uso de headers hembra para montar la Raspberry Pi Pico W facilitó la extracción y reprogramación del microcontrolador sin riesgo de sobrecalentamiento ni daño por soldadura directa.

- Se observó que el consumo conjunto del módulo WiFi y la matriz de 16 LEDs es significativo, lo que justifica el uso de una batería de 9 V de alta capacidad como fuente de alimentación externa.
- La disposición física de los LEDs en la maqueta debe respetar una polaridad común en las filas para simplificar el cableado y evitar cortocircuitos durante el ensamblaje.
- El uso de tubos termocontraíbles desde las primeras soldaduras mejoró el acabado estético y la integridad eléctrica de los cables de los LEDs, tal como se aplicó el 09/04/26.
- Se confirmó que pegar estereofon debajo de la protoboard y los LEDs actúa como aislante eléctrico y estabilizador mecánico, previniendo falsos contactos durante el transporte y uso del prototipo.
- El interruptor de 4 pines implementado el 4/05/26 permitió operar la maqueta autónomamente con la batería, eliminando la dependencia del cable USB al PC.
- Se comprobó que el Buzzer pasivo con señal PWM a 1 kHz (GP5) mejora la experiencia de usuario al ofrecer retroalimentación sonora inmediata durante la digitación Morse, facilitando el control de la duración de los pulsos.
- El uso de `time.ticks_ms()` y `time.ticks_diff()` resultó ser más preciso que `sleep()` para medir la duración de los pulsos Morse, pues no bloquea el hilo de ejecución y permite detectar el fin del pulso en tiempo real.
- La estructura de diccionarios en Python (`MORSE_DICC` y `PATRONES_LEDMAQUE`) optimizó el tiempo de búsqueda para la traducción de código Morse a caracteres y para el mapeo de cada letra a los bits de los registros de corrimiento.
- Los patrones del diccionario `PATRONES_LEDMAQUE` debieron ajustarse en múltiples oportunidades (27/04/26 y 30/04/26) para reflejar la configuración real del circuito,

especialmente por los pines defectuosos; esto demostrar la importancia de documentar los cambios de hardware en el código.

- El manejo de excepciones en el script de la Raspberry (bloque try/except/finally) evitó colapsos del sistema ante desconexiones repentinas del WiFi, cerrando el socket de forma ordenada.
- Se implementó una lógica de limpieza automática de la matriz de LEDs tras 3 segundos de inactividad; este mismo umbral de tiempo sirve como indicador de espacio entre palabras en el código Morse.
- La segmentación del código en funciones (iniciar_sistema, connectToWifi, actualizar_maqueta, sonar_buzzer) facilitó la depuración y permitió unificar el código de los talleres con la lógica propia del proyecto el 27/04/26.
- El flujo de trabajo con ramas en Git (Desarrollo_1 para la Raspberry e interfaz para el PC) permitió el desarrollo paralelo sin interferencias entre ambas partes del sistema, integrando todo en main en la fase final.
- La comunicación mediante Sockets TCP/IP sobre WiFi ofreció una latencia muy baja, adecuada para la respuesta en tiempo real del juego; el protocolo TCP garantizó que ningún símbolo Morse se perdiera durante la transmisión.
- Se validó que el protocolo TCP es preferible sobre UDP en este proyecto, ya que la integridad de cada letra transmitida es crítica para la correcta puntuación del juego.
- Se identificó que las paredes u otras señales en 2,4 GHz pueden afectar el rango de conexión de la Raspberry Pi Pico W; es necesario asegurar la visibilidad directa entre dispositivos o minimizar obstáculos.

- El uso de la opción `SO_REUSEADDR` en el servidor de la PC es indispensable para permitir reinicios rápidos de la aplicación sin bloqueos del puerto 8001.
- La identificación manual de la IP del servidor en la red local es mandatoria; se recomienda asignar una IP estática al equipo servidor para evitar que el cliente (Pico W) falle al conectarse tras un reinicio del router.
- El uso de hilos (threading) en la interfaz de PC fue esencial para que el socket no bloqueara el bucle principal de Tkinter, garantizando que la GUI permaneciera responsiva durante la espera de datos.
- El Modo Versus fomenta la interactividad al permitir que un jugador opere la maqueta física mientras el otro usa el teclado virtual de la PC, replicando la dinámica de comunicación secreta de la serie.
- El sistema de puntuación basado en coincidencia de caracteres (10 puntos por carácter correcto) resultó sencillo de implementar y comprensible para los usuarios durante las pruebas.
- La visualización en tiempo real de las letras recibidas en el Entry de Tkinter facilita que el jugador corrija su interpretación del código Morse mientras aún escucha o ve la señal.
- La adquisición de componentes presentó contratiempos: la Raspberry Pi Pico W era la última unidad disponible en la tienda, y el regulador de voltaje fue entregado de forma incorrecta en la primera compra (06/04/26). Planificar las compras con al menos dos semanas de anticipación es indispensable en futuros proyectos.
- La decisión de usar LEDs Jumbo direccionales requirió agregar material reflectante (papel celofán) en los orificios de la caja para que la luz fuera visible; este aspecto de diseño físico debe considerarse desde la etapa de planificación.

Consideraciones finales:**Recomendaciones:**

Para garantizar la integridad del sistema, se debe utilizar exclusivamente el pin 40 (VSYS) de la Raspberry Pi Pico W al emplear fuentes externas; este pin cuenta con un regulador interno que maneja de forma segura voltajes de hasta 5,5 V. El servidor en la computadora debe activarse antes que la maqueta para que el socket esté listo al momento de la conexión inalámbrica.

No conectar los registros de corrimiento SN74LS164N en cascada. La experiencia del 30/04/26 demostró que operar ambos registros de forma independiente, cada uno con su propio par de pines Data/Clock en la Raspberry, simplifica el código y elimina errores de señal.

Verificar el funcionamiento de cada pin de los registros SN74LS164N antes del ensamblaje final. En este proyecto se identificaron pines defectuosos de fábrica (Q4 del primer registro; Q1 y Q4 del segundo), lo que obligó a reasignar columnas y reescribir el diccionario de patrones.

Verificar que el botón pulsador reciba voltaje activo del pin 3,3VOut de la Raspberry; conectar solo una pata a GND sin llevar voltaje al pin es un error que impedirá cualquier detección, como ocurrió el 1/05/26 en este proyecto.

Usar cables sólidos de un solo hilo para las conexiones en la protoboard. Los cables flexibles multifilar, aunque más fáciles de doblar, dificultaron los ajustes y reacomodos del circuito el 28/04/26.

Proteger las soldaduras con tubos termocontraíbles desde el inicio del ensamblaje, no como paso final. Aplicarlos desde el 09/04/26 mejoró la integridad eléctrica y el acabado estético de los cables.

Agregar material reflectante (papel celofán o similar) en los orificios de los LEDs de la maqueta para mejorar la visibilidad. Los LEDs Jumbo direccionales emiten luz en un ángulo estrecho que, sin este recurso, resulta poco visible para el espectador.

Pegar estereofon bajo la protoboard como aislante eléctrico y estabilizador mecánico. Esta práctica previene falsos contactos durante el transporte y uso del prototipo.

Elegir un botón pulsador con casquillo grande desde el inicio. El botón original fue reemplazado el 4/05/26 por uno más visible y ergonómico, lo que mejoró considerablemente la experiencia al ingresar código Morse.

Planificar la adquisición de componentes con al menos dos semanas de anticipación respecto a la fecha límite. La Raspberry Pi Pico W era la última unidad disponible y el regulador fue entregado de forma incorrecta en la primera compra, generando retrasos en el cronograma.

Usar ramas de Git separadas para el desarrollo del hardware (código de la Raspberry) y del software (interfaz gráfica) desde el inicio del proyecto. Esta estrategia permitió el avance paralelo sin interferencias y facilitó la integración final.

Probar cada componente de forma individual antes de integrarlo al sistema completo. El LED quemado de la fila 2 y los pines defectuosos de los registros fueron detectados tardíamente, generando retrabajo que habría podido evitarse con pruebas unitarias tempranas.

Mantener la implementación de timeouts cortos en la escucha de datos de la Raspberry para que el dispositivo opere de forma no bloqueante, procesando datos entrantes sin perder la capacidad de detectar los pulsos del botón Morse.

Instalar un disipador de calor en el regulador L7805 o garantizar ventilación suficiente en la carcasa, dado que este componente disipa el exceso de energía como calor y puede dañar la protoboard en usos prolongados.

Asegurar las soldaduras e interconexiones con silicona caliente o cinta aislante para minimizar el riesgo de falsos contactos causados por vibraciones durante el transporte.

Es preferible el uso de baterías de Li-ion de alta capacidad sobre las alcalinas convencionales, ya que el consumo constante del módulo WiFi y la matriz de LEDs agota rápidamente estas últimas.

Durante el montaje, dejar aberturas en la carcasa cerca de la antena integrada de la Pico W para no obstruir la señal WiFi y mantener una comunicación estable.

Se recomienda el uso de librerías como Pygame para el desarrollo de interfaces gráficas futuras, aprovechando sus capacidades nativas para gestionar menús, animaciones y sonidos de forma más eficiente que Tkinter.

Migrar los diccionarios de patrones de LEDs (PATRONES_LEDMAQUE) a archivos JSON externos en versiones futuras para facilitar ajustes de mapeo sin necesidad de modificar el código fuente de la Raspberry.

Mantener la arquitectura de hilos (threading) en la gestión de red de la PC para prevenir el congelamiento de la interfaz gráfica mientras se esperan datos de la maqueta.

Etiquetar los cables internos del circuito con números o colores para facilitar las labores de mantenimiento, diagnóstico y reparación rápida del prototipo.

Análisis de resultados:

El sistema StrangerTec logró cumplir con los objetivos planteados de forma funcional. La integración entre el hardware empotrado y la interfaz gráfica de PC operó de manera estable

durante las pruebas finales, con una latencia de comunicación TCP/IP suficientemente baja para la respuesta en tiempo real que exige el juego.

En cuanto al hardware, la matriz de 16 LEDs respondió correctamente a los patrones enviados por los registros de corrimiento tras los ajustes realizados el 30/04/26, cuando se migró de una arquitectura en cascada a registros independientes. El primer registro controla 7 columnas y el segundo las 6 restantes, con la columna 5 asignada al pin 13 y la columna 9 al pin 12 del segundo registro SN74LS164N, debido a que los pines Q4 del primer registro y Q1 y Q4 del segundo resultaron defectuosos de fábrica. Una vez aplicados estos ajustes, la visualización de letras y números en la maqueta fue precisa y consistente.

La detección de pulsos Morse mediante el botón funcionó correctamente una vez que se resolvió el problema de alimentación del 1/05/26, donde se determinó que el pin GP16 no recibía los 3,3 V necesarios. Al conectar una pata del botón al pin 3,3VOut de la Raspberry, la clasificación de puntos (presiones menores a 300 ms) y rayas (presiones mayores o iguales a 300 ms) operó de forma estable y sin detecciones falsas. El buzzer pasivo en GP5 proporcionó retroalimentación auditiva inmediata y sincronizada con cada pulso, lo que facilitó el control de la duración de las señales durante las pruebas de usuario.

El sistema de alimentación autónoma, implementado el 4/05/26 con la batería de 9 V, el interruptor y el regulador L7805 conectado al pin VSYS, permitió operar la maqueta sin depender del cable USB, cumpliendo con el requisito de movilidad del prototipo.

En el software de la Raspberry Pi Pico W, el uso de `time.ticks_ms()` y `time.ticks_diff()` demostró ser más preciso que `sleep()` para la medición de pulsos, y el manejo de excepciones garantizó que desconexiones inesperadas del WiFi no colapsaran el sistema. La lógica de limpieza automática de la matriz tras 3 segundos de inactividad funcionó también como indicador de espacio entre palabras, cumpliendo una doble función dentro del flujo del juego.

En la interfaz gráfica del PC, el servidor multihilo permitió que Tkinter permaneciera responsivo mientras se recibían datos de la maqueta. El modo de 1 jugador y el modo Versus operaron correctamente, con el sistema de puntuación asignando 10 puntos por carácter coincidente. La visualización en tiempo real de las letras recibidas en el Entry fue validada como útil para que el jugador corrigiera su interpretación durante la partida.

Como resultado general, el proyecto demostró que es posible construir un sistema empotrado funcional de traducción Morse con comunicación inalámbrica, visualización LED y retroalimentación auditiva, empleando componentes de bajo costo y programación en MicroPython y Python estándar.

Diagrama de arquitectura:

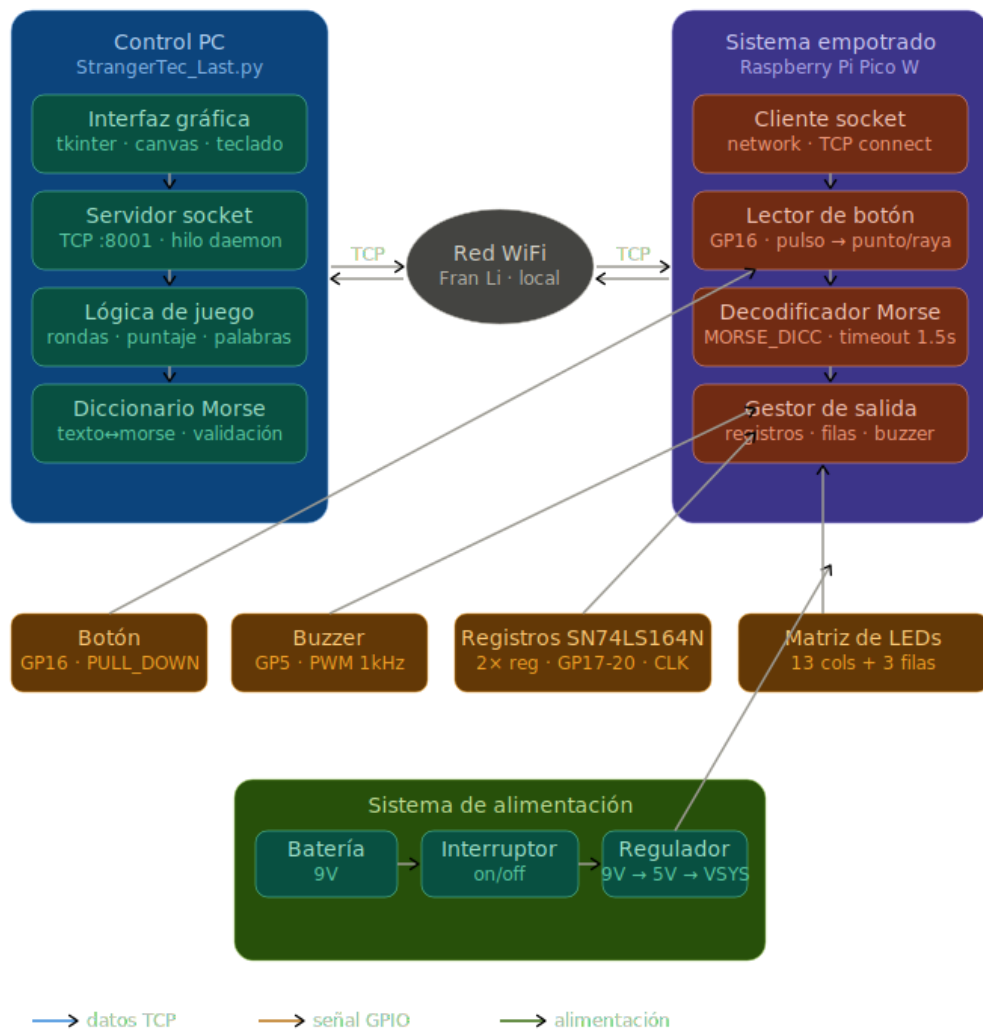
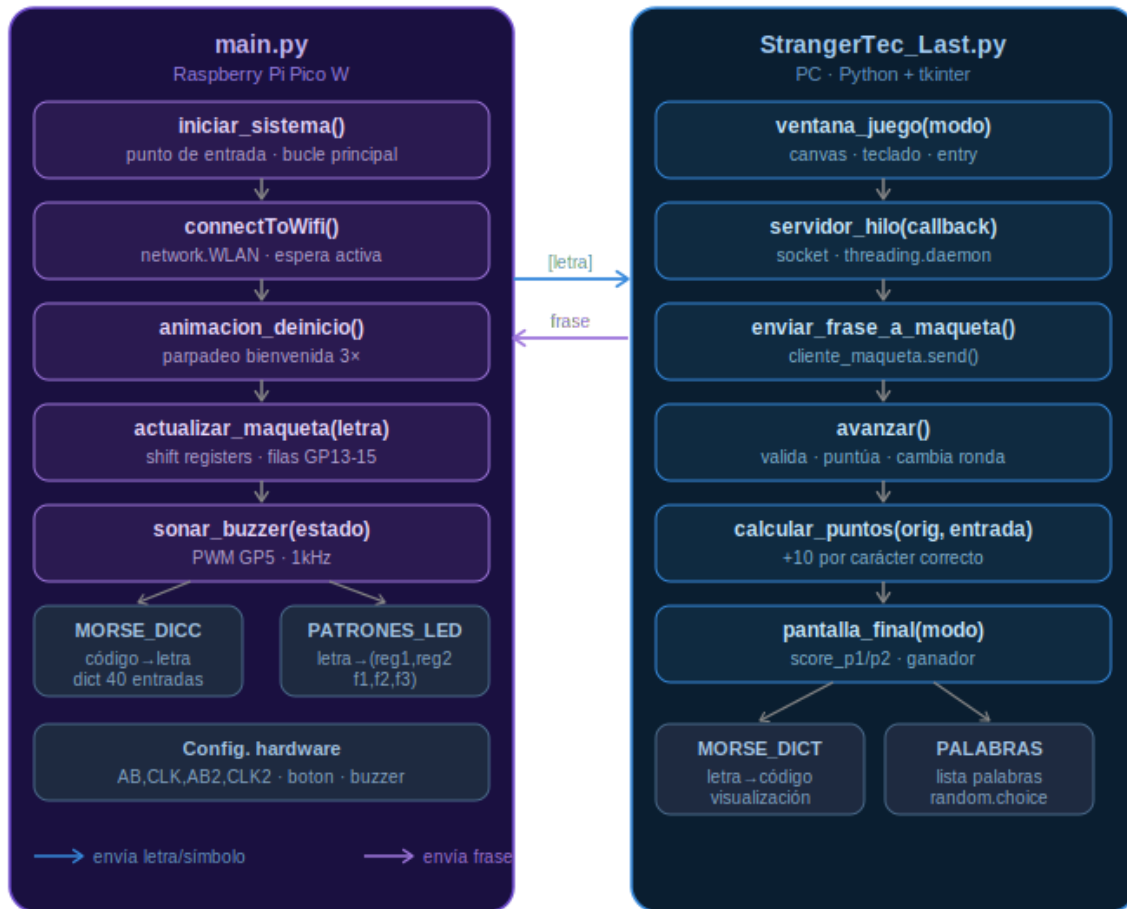


Diagrama de módulos del sistema:



Literatura y fuentes consultadas (referencias) :

Google. (2026). *Gemini* (versión Gemini 2.0 Flash) [Modelo de lenguaje de IA]. <https://gemini.google.com>

Gutiérrez Moreira, E. (2026). *Emanuel-Gutiérrez-Moreira* [Repositorio personal]. GitHub. <https://github.com/emagzmo/Emanuel-Guti-rrez-Moreira>

MicroPython. (s.f.). *MicroPython documentation*. <https://micropython.org>

Morse Code World. (s.f.). *Alfabeto Morse y espaciado*. <https://morsecw.com/alfabeto.html#espaciado>

Raspberry Pi Ltd. (s.f.). *Raspberry Pi Pico W datasheet*. Raspberry Pi Documentation. <https://www.raspberrypi.com/documentation/microcontrollers/raspberry-pi-pico.html>

- Texas Instruments. (2015). *SN74LS164 8-bit parallel-out serial shift registers* (Rev. C) [Hoja de datos]. <https://www.ti.com/lit/ds/symlink/sn74ls164.pdf>
- STMicroelectronics. (2013). *L78xx voltage regulators* [Hoja de datos]. <https://www.st.com/resource/en/datasheet/17805.pdf>
- Python Software Foundation. (2024). *tkinter — Python interface to Tcl/Tk*. Python 3 Documentation. <https://docs.python.org/3/library/tkinter.html>